

claude-wsl-ubuntu / 4f364eb7-a600-4a2c-9b37-e5a313b6db9d

Metadata

```
- Source: `claude-wsl-ubuntu`
- Kind: `claude`
- Source file: `\\wsl.localhost\Ubuntu\home\avidullu\.claude\projects\home-avidullu-projects-khelsutra-guru\4f364eb7-a600-4a2c-9b37-e5a313b6db9d\subagents\workflows\wf_fa7a2077-3a9\agent-a14d9ffbcd53d0e5.jsonl`
- SHA-256: `5c6f0c4a4046bec7151de7d09678035575a653b3f18d09401a47b861da9c74aa`
- Source modified: `2026-06-29T10:51:42+00:00`
- Imported at: `2026-07-05T16:48:30+00:00`
- project: `wf_fa7a2077-3a9`
- session_id: `4f364eb7-a600-4a2c-9b37-e5a313b6db9d`
```

Transcript

1. user (2026-06-29T10:50:03.896Z)

Read these code anchors in /home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer and report each one's CURRENT state with file:line references:

1. backend/pipeline/detectors/tracknet_runner.py :: _trim_inactive_tail ? how the R4 rally-end trim works and how it is gated on temporal_density.
2. backend/config/models.py ? the field inactivity_temporal_density (confirm current default value, exact line, and the surrounding config class name).
3. backend/eval/rally_seg_eval.py :: _build_preset and the --inactivity-temporal-density / --vs-inactivity-temporal-density CLI flags ? how the A/B is wired.
4. backend/eval/windowing.py ? how WindowingPreset carries the flag.
5. backend/eval/promotion.py :: promote_if_served_safe ? the served-safety gate logic and its served_regress_eps default.

Quote the exact current default of inactivity_temporal_density. Return data, not prose.

2. assistant (2026-06-29T10:50:06.357Z)

I'll read the code anchors. Let me start by locating the files.

3. user (2026-06-29T10:50:07.865Z)

```
/home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer/backend/config/models.py
/home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/rally_seg_eval.py
/home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/promotion.py
/home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/windowing.py
/home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/detectors/tracknet_runner.py
```

4. assistant (2026-06-29T10:50:09.262Z)

All five files exist. Let me read them.

5. user (2026-06-29T10:50:10.720Z)

```
329:         inactivity_temporal_density: bool = False,
359:         ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on every
gap-less
360:         over-cap window and over-segments normal clips), this runs AFTER the R4 inactivity
trim and
413:         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so each
rally
421:         TrackNetRunner._trim_inactive_tail(
426:             temporal_density=inactivity_temporal_density,
431:         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
```

```

trim, any
450:     def _trim_inactive_tail(
455:         temporal_density: bool = False,
457:         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
467:         ``temporal_density`` (default False = legacy/promoted behaviour) selects the density
474:         at the SERVED operating point (R4 is currently default-ON in prod; see
RALLY_DETECTION_FIXES_PLAN)."""
487:         if temporal_density:
556:         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split AND
the R4

```

6. user (2026-06-29T10:50:12.788Z)

```

18:class ValidationRelevanceConfig(BaseModel):
24:class ValidationConfig(BaseModel):
46:class TrackNetConfig(BaseModel):
67:class WasbIndexConfig(BaseModel):
112:class FusionConfig(BaseModel):
146:class IndexingConfig(BaseModel):
199:    inactivity_temporal_density: bool = False
265:class OutputConfig(BaseModel):
273:class WatermarkConfig(BaseModel):
295:class StitchingConfig(BaseModel):
311:class LoggingConfig(BaseModel):
321:class SecurityConfig(BaseModel):
354:class StorageConfig(BaseModel):
395:class DeploymentConfig(BaseModel):
415:class BillingConfig(BaseModel):
436:class FlywheelConfig(BaseModel):
458:class AppConfig(BaseModel):

```

7. user (2026-06-29T10:50:16.300Z)

```

405         if max_window_duration > 0:
406             groups = TrackNetRunner._split_overlong_groups(
407                 groups,
408                 int(max_window_duration * fps),
409                 int(min_split_gap * fps),
410                 hard_cap=window_hard_cap,
411             )
412
413         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
414         # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
415         # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
416         # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
417         # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
418         if window_inactivity_end > 0:
419             tw = int(window_inactivity_end * fps)
420             groups = [
421                 TrackNetRunner._trim_inactive_tail(
422                     g,
423                     tw,
424                     inactivity_min_density,
425                     inactivity_rally_thresh,
426                     temporal_density=inactivity_temporal_density,
427                 )
428                 for g in groups
429             ]
430

```

```

431         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
432         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
433         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
434         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
435         forced_flags = [False] * len(groups)
436         if window_blob_fallback and max_window_duration > 0:
437             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
438                 groups, int(max_window_duration * fps), window_blob_factor
439             )
440
441         windows = []
442         for g, forced in zip(groups, forced_flags):
443             w = _finalize(g)
444             if forced:
445                 w["forced_cut"] = True
446             windows.append(w)
447         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
448
449     @staticmethod
450     def _trim_inactive_tail(
451         group: List[Tuple[int, float]],
452         trail_frames: int,
453         min_density: float,
454         rally_thresh: float,
455         temporal_density: bool = False,
456     ) -> List[Tuple[int, float]]:
457         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
458         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
459         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
460         (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
461         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
462         LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
463         can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
464         exists at all the group is kept whole (fail-safe, no over-trim). 0(n) two-
pointer; the START
465         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding).
466
467         ``temporal_density`` (default False = legacy/promoted behaviour) selects the
density
468         DENOMINATOR (code-review finding B). ``group`` carries only active frames, so
the legacy
469         denominator ? the COUNT of active samples in the trailing window (``i - lo +
1``) ? reads a
470         lone fast sample after a long tracking gap as 100% dense and holds the tail
open. When True,
471         the denominator is the trailing window's TEMPORAL length instead, so sparse
tracking can't
472         inflate density. With DENSE tracking the two are identical, so this is a no-op
on
473         well-tracked clips ? hence it ships flag-gated default-OFF until measured on the
golden set
474         at the SERVED operating point (R4 is currently default-ON in prod; see
RALLY_DETECTION_FIXES_PLAN)."""
475         if trail_frames <= 0 or len(group) < 2:

```

```

476         return group
477     frames = [f for f, _ in group]
478     fast = [1 if v > rally_thresh else 0 for _, v in group]
479     lo = 0
480     fast_sum = 0
481     last_dense = -1
482     for i in range(len(frames)):
483         fast_sum += fast[i]
484         while frames[lo] <= frames[i] - trail_frames:
485             fast_sum -= fast[lo]
486             lo += 1
487         if temporal_density:
488             # Trailing window's TEMPORAL length: full ``trail_frames`` once that
much footage
489             # has elapsed, else the footage seen so far. Sparse tracking can't
inflate it.
490             denom = min(trail_frames, frames[i] - frames[0] + 1)
491         else:
492             denom = (
493                 i - lo + 1
494             ) # legacy: count of active samples in the trailing window
495         if denom > 0 and fast_sum / denom >= min_density:
496             last_dense = i
497     return group[: last_dense + 1] if last_dense >= 0 else group
498
499     @staticmethod
500     def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:
501         """Index of the active frame nearest the group's temporal midpoint (used to chop
a
502         gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
fallback."""
503         mid = (group[0][0] + group[-1][0]) / 2.0
504         split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
505         return split_i
506
507     @staticmethod
508     def _split_overlong_groups(
509         groups: List[List[Tuple[int, float]]],
510         max_win_frames: int,
511         min_split_frames: int,
512         hard_cap: bool = False,
513     ) -> List[List[Tuple[int, float]]]:
514         """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
515         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
516         boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
517         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
518         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
519         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
520         bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
521         if max_win_frames <= 0:
522             return groups
523         out: List[List[Tuple[int, float]]] = []
524         work = list(groups)
525         while work:
526             g = work.pop()
527             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
528                 out.append(g)
529                 continue

```

```

530         best_i, best_gap = -1, -1
531         for i in range(1, len(g)):
532             gap = g[i][0] - g[i - 1][0]
533             if gap > best_gap:
534                 best_gap, best_i = gap, i

```

8. user (2026-06-29T10:50:16.982Z)

```

146     class IndexingConfig(BaseModel):
147         default_segmenter: str = "yolo_hybrid"
148         use_gpu: bool = True
149         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
150         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
151         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
152         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
153         # RALLY_DETECTOR_IMPL env var overrides this. See
154         pipeline/detectors/{stub,native_wasb}_runner.py.
155         detector_impl: str = "auto"
156         motion_threshold: float = 0.08
157         min_segment_duration: float = 3.0
158         dead_time_merge_gap: float = 2.0
159         chunk_duration: float = 90.0
160         chunk_overlap: float = 10.0
161         detector_model: str = "fasterrcnn_resnet50_fpn"
162         detector_score_threshold: float = 0.5
163         yolo_velocity_threshold: float = 0.15
164         yolo_frame_skip: int = 3
165         yolo_tracker: str = "bytetrack.yaml"
166         yolo_prefetch_workers: int = 2
167         min_action_window_duration: float = 2.0
168         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
169         = a window
170         # longer than this many seconds is recursively split at its largest internal
171         inactivity gap (?
172         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
173         swallow
174         # several rallies + the dead time between them. Never merges, only cuts (recall
175         can't drop).
176         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
177         Measured n=13
178         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
179         p=0.0005; cap=6
180         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
181         0.651?0.161). ***
182         max_window_duration: float = 6.0
183         window_min_split_gap: float = (
184             0.5 # min internal gap (s) that qualifies as a split point
185         )
186         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
187         inactivity gap
188         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
189         is
190         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
191         bound.
192         # Only cuts (recall can't drop). OFF by default until measured/promoted.
193         window_hard_cap: bool = False
194         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
195         slowly
196         # (picked up / walked) above velocity_thresh, so the window over-extends its END
197         (the measured
198         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
199         trailing
200         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
201         cuts.
202         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but

```

```

significant
188      # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
189      # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
190      window_inactivity_end: float = 1.5
191      inactivity_rally_thresh: float = 0.20
192      inactivity_min_density: float = 0.30
193      # R4 density DENOMINATOR fix (code-review finding B), default OFF until measured at
the SERVED
194      # operating point. The R4 trim's fast-fraction denominator is the COUNT of active
samples in the
195      # trailing window, which sparse tracking inflates (one fast sample after a gap ?
100% dense ?
196      # tail held open). When True, the denominator is the window's TEMPORAL length
instead. No-op
197      # under dense tracking (sample-count == temporal-span), so this can flip with zero
regression on
198      # well-tracked clips; flag-gated default-OFF since R4 itself is default-ON. See
QUALITY_ITERATIONS.
199      inactivity_temporal_density: bool = False
200      # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
201      # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
202      # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
203      # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
204      # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
205      # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
206      # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
207      # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
208      window_blob_fallback: bool = False
209      # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
210      # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
211      # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
212      # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
213      window_blob_factor: float = 4.0
214      log_yolo_candidates: bool = True
215      store_yolo_candidates: bool = True
216      ai_handoff_padding: float = 2.0
217      ai_max_workers: int = 5
218      # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
219      # chunks of a high-bitrate (4K) source blow past the provider upload cap
220      # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as

```

9. assistant (2026-06-29T10:50:19.272Z)

Now let me read the eval files.

10. user (2026-06-29T10:50:21.751Z)

```

574: def _build_preset(args: argparse.Namespace, side: str) -> WindowingPreset:
587:     temporal_density = args.inactivity_temporal_density
603:     args.vs_inactivity_temporal_density or args.inactivity_temporal_density
612:     preset = replace(preset, inactivity_temporal_density=True)
624: def main() -> None:

```

```

628:     ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
629:     ap.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR)
630:     ap.add_argument(
633:     ap.add_argument("--vs", default=None, help="second preset to diff against --preset")
634:     ap.add_argument("--iou", type=float, default=DEFAULT_IΟΥ)
635:     ap.add_argument(
641:     ap.add_argument(
647:     ap.add_argument(
653:     ap.add_argument(
660:     ap.add_argument(
666:     ap.add_argument(
671:     ap.add_argument(
677:     ap.add_argument(
682:     ap.add_argument(
688:     ap.add_argument(
694:     ap.add_argument(
700:     ap.add_argument(
706:     ap.add_argument(
712:     ap.add_argument(
713:         "--inactivity-temporal-density",
718:     ap.add_argument(
719:         "--vs-inactivity-temporal-density",
721:         help="R4 temporal-density on --vs (default: same as --inactivity-temporal-density);
use "
722:         "'--preset served --vs served --vs-inactivity-temporal-density' for the P2 promotion
A/B",
724:     ap.add_argument(
730:     ap.add_argument(
736:     ap.add_argument(
742:     ap.add_argument(
748:     ap.add_argument(
754:     ap.add_argument(
763:     ap.add_argument(
770:     ap.add_argument(
777:     ap.add_argument(
783:     base = _build_preset(args, "base")
786:     cand = _build_preset(args, "cand")

```

11. user (2026-06-29T10:50:23.488Z)

```

48:@dataclass(frozen=True)
49:class WindowingPreset:
63:    #: inactivity gap ? the bounded-windowing over-merge fix (W1, RALLY_QUALITY_RESEARCH.md
$6).
66:    #: inactivity gap is chopped at its midpoint until ? the cap (continuously-active fix).
OFF by default.
68:    #: R4 rally-END inactivity trim (s, 0 = OFF): trim a window's post-rally slow-drift tail
back to
70:    inactivity_end: float = 0.0
72:    inactivity_rally_thresh: float = 0.08
73:    inactivity_min_density: float = 0.34
77:    inactivity_temporal_density: bool = False
85:    def as_tuple(self) -> Tuple[float, float, float]:
91:    def to_dict(self) -> Dict[str, Any]:
99:        "inactivity_end": self.inactivity_end,
100:        "inactivity_rally_thresh": self.inactivity_rally_thresh,
101:        "inactivity_min_density": self.inactivity_min_density,
102:        "inactivity_temporal_density": self.inactivity_temporal_density,
108:    def _served_preset_from_defaults() -> WindowingPreset:
132:        inactivity_end=float(
133:            idx.window_inactivity_end
135:        inactivity_rally_thresh=float(idx.inactivity_rally_thresh),
136:        inactivity_min_density=float(idx.inactivity_min_density),
137:        inactivity_temporal_density=bool(
138:            idx.inactivity_temporal_density

```

```

161: def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
185:     inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
186:     inactivity_rally_thresh=float(
187:         idx.get("inactivity_rally_thresh", SERVED.inactivity_rally_thresh)
189:     inactivity_min_density=float(
190:         idx.get("inactivity_min_density", SERVED.inactivity_min_density)
192:     inactivity_temporal_density=bool(
193:         idx.get("inactivity_temporal_density", SERVED.inactivity_temporal_density)
200: def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
214: def build_windows(
234:     window_inactivity_end=preset.inactivity_end,
235:     inactivity_rally_thresh=preset.inactivity_rally_thresh,
236:     inactivity_min_density=preset.inactivity_min_density,
237:     inactivity_temporal_density=preset.inactivity_temporal_density,
243: def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:

```

12. user (2026-06-29T10:50:24.259Z)

```

1: """Promotion gate with the eval?serve guard: a cue is promotable ONLY if it does not regress
9: regressing on the served one (**?0.008**). Grading harder isn't the fix; **measuring on the
10: served windowing** is.
16: REGRESS (its served paired ?F1 mean ? ``-served_regress_eps``, and ? when its served CI
is
17: estimable ? that CI's lower bound does not sit clearly in regression territory).
20: can never carry a promotion on its own. The decision returns BOTH the served and the proposal
24: paths are untouched until a caller routes its A/B through :func:`promote_if_served_safe`.
35: __all__ = ["ServedGateResult", "promote_if_served_safe", "served_regresses"]
38: def served_regresses(
46:     """Does variant B REGRESS variant A at the served windowing? (the eval?serve guard core).
50:     a single ``regresses`` boolean:
52:     - ``regresses=True`` when the served mean ?F1 is below ``-eps`` (a real served drop),
OR
53:     the served ?F1 CI lies **entirely** below ``-eps`` (a confidently-negative band).
54:     - ``regresses=False`` when the cue holds the line at the served operating point (mean ?
57:     ``eps`` is the regression tolerance: 0.0 = "must not drop at all on the served mean". A
tiny
64:     # Regression if the served mean dropped beyond tolerance, OR the whole CI is net-
negative.
65:     mean_regresses = mean_delta < -eps
67:     regresses = bool(mean_regresses or ci_confidently_negative)
69:     "regresses": regresses,
82:     """A promotion decision that BOTH measures the cue on the served windowing AND reports
the
86:     *and* the served check did not veto (the cue does not regress at the served windowing).
87:     - ``served_safe`` ? the eval?serve guard's verdict in isolation (cue does not regress
served).
88:     - ``vetoed_by_served`` ? ``True`` when the proposal/base gate WOULD have promoted but the
89:     served regression check overruled it (the exact Gate-A failure mode this module
prevents).
91:     *promotion-grade* scores (served by default ? see :func:`promote_if_served_safe`).
92:     - ``served`` / ``proposal`` ? the paired B?A summary at each windowing (means + CIs +
sign
93:     tests), so a reviewer sees "+0.074 proposal / ?0.008 served" side by side.
98:     served_safe: bool
99:     vetoed_by_served: bool
102:     served: Dict[str, Any]
105:     def as_dict(self) -> Dict[str, Any]:
108:         "served_safe": self.served_safe,
109:         "vetoed_by_served": self.vetoed_by_served,
112:         "served": self.served,
117: def promote_if_served_safe(
118:     served_a: Sequence[float],
119:     served_b: Sequence[float],
126:     served_regress_eps: float = 0.0,
130:     """Promote a cue ONLY if it does not regress at the SERVED windowing (the foundational

```



```

rule).
133:    applied to the **served** scores ? the operating point production runs ? so a proposal-
only
139:    served_a/served_b: per-video held-out scores for variant A (control) and B (the cue),
142:    contrast; the served regression guard still has the final say.
145:    the served default if it regresses served. So for ``structural=True`` we report
146:    ``keep_on`` honestly but ``promote`` reflects the served-safety check.
147:    served_regress_eps: served regression tolerance (0.0 = "no served-mean drop allowed").
153:    served_a,
154:    served_b,
162:    # 2) The eval?serve guard: does B regress A at the served windowing?
163:    served = served_regresses(
164:        served_a, served_b, eps=served_regress_eps, n_boot=n_boot, seed=seed
166:    served_safe = not served["regresses"]
171:    proposal = served_regresses(
172:        proposal_a, proposal_b, eps=served_regress_eps, n_boot=n_boot, seed=seed
175:    # 4) Compose the verdict. The served guard can only ever REMOVE a promotion, never add
one.
178:    # it is only auto-promoted to the SERVED default when it is served-safe. If it
regresses
179:    # served, promote=False (don't bake an eval-only win into production) while keep_on
holds.
180:    promote = served_safe
182:    if served_safe:
184:        "structural guard kept ON; served-safe (does not regress at the served "
185:        f"windowing, ?F1 {served['mean_delta']:+.4f}) ? eligible for served default"
189:        "structural guard kept ON, but NOT promoted to the served default: it "
190:        f"REGRESSES at the served windowing (?F1 {served['mean_delta']:+.4f}); "
191:        "an eval-only win must not be served"
195:    if would_promote and not served_safe:
199:        "VETOED by the served-windowing guard: the cue clears the honest bar but "
200:        f"REGRESSES at the served windowing (served ?F1 {served['mean_delta']:+.4f})"
201:        f", CI [{served['ci_low']:+.4f}, {served['ci_high']:+.4f}]. A win on a "
208:        "promoted: clears the honest bar AND is served-safe (served ?F1 "
209:        f"{served['mean_delta']:+.4f}). " + base.reason
218:    served_safe=served_safe,
219:    vetoed_by_served=vetoed,
222:    served=served,

```

13. assistant (2026-06-29T10:50:26.249Z)

Now let me read the specific sections in detail for the eval files.

14. user (2026-06-29T10:50:28.436Z)

```

574    def _build_preset(args: argparse.Namespace, side: str) -> WindowingPreset:
575        """Resolve the ``base`` (``--preset``) or ``cand`` (``--vs``) windowing from
``args`` by
576        applying the ``dataclasses.replace`` overrides for cap / hard-cap / blob-fallback /
577        inactivity. ``cand`` falls back to the base override when its ``--vs-*`` variant is
unset
578        (so a baseline-vs-candidate promotion check runs in one shot)."""
579        from dataclasses import replace
580
581        if side == "base":
582            preset = _resolve_preset(args.preset)
583            cap = args.max_window_duration
584            hard_cap = args.hard_cap
585            blob_fallback = args.blob_fallback
586            inactivity_end = args.inactivity_end
587            temporal_density = args.inactivity_temporal_density
588        else:
589            preset = _resolve_preset(args.vs)
590            cap = (
591                args.vs_max_window_duration

```

```

592         if args.vs_max_window_duration is not None
593         else args.max_window_duration
594     )
595     hard_cap = args.vs_hard_cap or args.hard_cap
596     blob_fallback = args.vs_blob_fallback or args.blob_fallback
597     inactivity_end = (
598         args.vs_inactivity_end
599         if args.vs_inactivity_end is not None
600         else args.inactivity_end
601     )
602     temporal_density = (
603         args.vs_inactivity_temporal_density or args.inactivity_temporal_density
604     )
605     if cap is not None:
606         preset = replace(preset, max_window_duration=cap)
607     if hard_cap:
608         preset = replace(preset, hard_cap=True)
609     if blob_fallback:
610         preset = replace(preset, blob_fallback=True)
611     if temporal_density:
612         preset = replace(preset, inactivity_temporal_density=True)
613     if args.blob_factor is not None:
614         preset = replace(preset, blob_factor=args.blob_factor)
615     if inactivity_end is not None:
616         preset = replace(preset, inactivity_end=inactivity_end)
617     if args.inactivity_rally_thresh is not None:
618         preset = replace(preset, inactivity_rally_thresh=args.inactivity_rally_thresh)
619     if args.inactivity_min_density is not None:
620         preset = replace(preset, inactivity_min_density=args.inactivity_min_density)
621     return preset
622
623

```

15. user (2026-06-29T10:50:29.289Z)

```

706     ap.add_argument(
707         "--inactivity-min-density",
708         type=float,
709         default=None,
710         help="R4 min trailing fast-fraction to stay 'in rally' (both presets; default
0.34)",
711     )
712     ap.add_argument(
713         "--inactivity-temporal-density",
714         action="store_true",
715         help="R4 density over the trailing window's TEMPORAL length, not the active-
sample "
716         "count, on --preset (finding-B fix; no-op under dense tracking, trims sparse-
track tails)",
717     )
718     ap.add_argument(
719         "--vs-inactivity-temporal-density",
720         action="store_true",
721         help="R4 temporal-density on --vs (default: same as --inactivity-temporal-
density); use "
722         "'--preset served --vs served --vs-inactivity-temporal-density' for the P2
promotion A/B",
723     )
724     ap.add_argument(
725         "--tau-start",
726         type=float,
727         default=None,
728         help="R2 tolerance-match start window (s); default 2.0 (forgives the service
window)",
729     )

```

730 ap.add_argument(

16. user (2026-06-29T10:50:29.972Z)

```
48     @dataclass(frozen=True)
49     class WindowingPreset:
50         """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
51         point.
52         ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
53         counts as "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
54         window;
55         ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
56         the three knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
57         windowing.
58         """
59         name: str
60         velocity_thresh: float
61         merge_gap: float
62         min_window_duration: float
63         #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
64         internal
65         #: inactivity gap ? the bounded-windowing over-merge fix (W1,
66         RALLY_QUALITY_RESEARCH.md §6).
67         max_window_duration: float = 0.0
68         #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
69         internal
70         #: inactivity gap is chopped at its midpoint until ? the cap (continuously-active
71         fix). OFF by default.
72         hard_cap: bool = False
73         #: R4 rally-END inactivity trim (s, 0 = OFF): trim a window's post-rally slow-drift
74         tail back to
75         #: the last frame whose trailing window stays dense with FAST motion. The measured
76         rally-end fix.
77         inactivity_end: float = 0.0
78         #: velocity above which motion counts as "rally" (vs slow drift) for the R4 trim;
79         min fast-fraction.
80         inactivity_rally_thresh: float = 0.08
81         inactivity_min_density: float = 0.34
82         #: R4 density-denominator fix (finding B), default OFF. When True the R4 fast-
83         fraction is taken
84         #: over the trailing window's TEMPORAL length, not the count of active samples
85         (which sparse
86         #: tracking inflates). No-op under dense tracking; gated default-OFF until measured
87         on SERVED.
88         inactivity_temporal_density: bool = False
89         #: R5 blob-fallback (default OFF). After the cap split + R4 trim, midpoint-chop any
90         group still
91         #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived
92         blob ?
93         #: mahadevpura-1's ~11× residual) and flag it. Runs AFTER R4 (vs ``hard_cap``
94         before) and the
95         #: factor keeps it surgical (a normal long rally is left alone).
96         blob_fallback: bool = False
97         blob_factor: float = 4.0
98
99         def as_tuple(self) -> Tuple[float, float, float]:
100             """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
101             shape the
102             ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
103             (The
```

17. assistant (2026-06-29T10:50:32.221Z)

Now let me read the `promote_if_served_safe` function signature and the `served_regress_eps` default precisely.

18. user (2026-06-29T10:50:33.683Z)

```
117     def promote_if_served_safe(  
118         served_a: Sequence[float],  
119         served_b: Sequence[float],  
120         *,  
121         proposal_a: Optional[Sequence[float]] = None,  
122         proposal_b: Optional[Sequence[float]] = None,  
123         structural: bool = False,  
124         min_n: int = 5,  
125         p_threshold: float = 0.05,  
126         served_regress_eps: float = 0.0,  
127         n_boot: int = 2000,  
128         seed: int = 0,  
129     ) -> ServedGateResult:  
130         """Promote a cue ONLY if it does not regress at the SERVED windowing (the  
foundational rule).  
131  
132         The honest bar (`should_promote_for_user`: min-N floor + ?F1 CI excludes 0 + sign  
test) is  
133         applied to the **served** scores ? the operating point production runs ? so a  
proposal-only  
134         win can never carry a promotion. The optional `proposal_` scores are scored too  
and  
135         reported alongside, purely to surface the eval?serve contrast (they NEVER promote on  
their  
136         own).  
137  
138         Args:  
139             served_a/served_b: per-video held-out scores for variant A (control) and B (the  
cue),  
140             measured at the **SERVED** windowing. These drive the decision.  
141             proposal_a/proposal_b: the same scores at the PROPOSAL windowing (optional).  
Reported for  
142             contrast; the served regression guard still has the final say.  
143             structural: forwarded to `should_promote_for_user` ? a structural guard (e.g.  
Gate-A vs  
144             held-shuttle FPs) is never *disabled* on "no lift", but it is ALSO not  
auto-*promoted* to  
145             the served default if it regresses served. So for `structural=True` we report  
146             `keep_on` honestly but `promote` reflects the served-safety check.  
147             served_regress_eps: served regression tolerance (0.0 = "no served-mean drop  
allowed").  
148  
149             Returns a :class:`ServedGateResult` with the final verdict + both windowings'  
evidence.  
150             """  
151             # 1) The honest gate, applied to the SERVED scores (never the proposal ones).  
152             base = should_promote_for_user(  
153                 served_a,  
154                 served_b,  
155                 structural=structural,  
156                 min_n=min_n,  
157                 p_threshold=p_threshold,  
158                 n_boot=n_boot,  
159                 seed=seed,  
160             )  
161  
162             # 2) The eval?serve guard: does B regress A at the served windowing?  
163             served = served_regresses(  

```

```

164         served_a, served_b, eps=served_regress_eps, n_boot=n_boot, seed=seed
165     )
166     served_safe = not served["regresses"]
167
168     # 3) Optional proposal-windowing view (reported, never promoting on its own).
169     proposal: Optional[Dict[str, Any]] = None
170     if proposal_a is not None and proposal_b is not None:
171         proposal = served_regresses(
172             proposal_a, proposal_b, eps=served_regress_eps, n_boot=n_boot, seed=seed
173         )
174
175     # 4) Compose the verdict. The served guard can only ever REMOVE a promotion, never
176     add one.
177     if structural:
178         # A structural guard ships ON regardless of measured lift (base.keep_on stays
179         True), but
180         # it is only auto-promoted to the SERVED default when it is served-safe. If it
181         regresses
182         # served, promote=False (don't bake an eval-only win into production) while
183         keep_on holds.
184         promote = served_safe
185         vetoed = False
186         if served_safe:
187             reason = (
188                 "structural guard kept ON; served-safe (does not regress at the served "
189                 f"windowing, ?F1 {served['mean_delta']:+.4f}) ? eligible for served
190                 default"
191             )
192         else:
193             reason = (
194                 "structural guard kept ON, but NOT promoted to the served default: it "
195                 f"REGRESSES at the served windowing (?F1 {served['mean_delta']:+.4f}); "
196                 "an eval-only win must not be served"
197             )
198     else:
199         would_promote = base.promote
200         if would_promote and not served_safe:
201             promote = False
202             vetoed = True
203             reason = (
204                 "VETOED by the served-windowing guard: the cue clears the honest bar but
205                 "
206                 f"REGRESSES at the served windowing (served ?F1
207                 {served['mean_delta']:+.4f})"
208                 f", CI [{served['ci_low']:+.4f}, {served['ci_high']:+.4f})). A win on a
209                 "
210                 "windowing production doesn't serve cannot promote."
211             )
212         elif would_promote:
213             promote = True
214             vetoed = False
215             reason = (
216                 "promoted: clears the honest bar AND is served-safe (served ?F1 "
217                 f"{served['mean_delta']:+.4f}). " + base.reason
218             )
219         else:
220             promote = False
221             vetoed = False
222             reason = base.reason
223
224     return ServedGateResult(
225         promote=promote,
226         served_safe=served_safe,
227         vetoed_by_served=vetoed,
228         reason=reason,

```

```
221         base_decision=base,
222         served=served,
223         proposal=proposal,
224     )
225
```

19. assistant (2026-06-29T10:50:37.298Z)

I now have all the data. Let me also confirm the `served\_regresses` eps default and verify the build_windows wiring in windowing.py at line 234-237.

20. user (2026-06-29T10:50:38.908Z)

```
38     def served_regresses(
39         a_scores: Sequence[float],
40         b_scores: Sequence[float],
41         *,
42         eps: float = 0.0,
43         n_boot: int = 2000,
44         seed: int = 0,
45     ) -> Dict[str, Any]:
46         """Does variant B REGRESS variant A at the served windowing? (the eval?serve guard
47         core).
48         ``a_scores``/``b_scores`` are per-video held-out scores **measured at the SERVED
49         windowing**
50         (aligned by video). Returns the paired B?A summary (mean ?F1 + bootstrap CI + sign
51         test) plus
52         a single ``regresses`` boolean:
53         - ``regresses=True`` when the served mean ?F1 is below ``-eps`` (a real served
54         drop), OR
```

21. assistant (2026-06-29T10:50:43.048Z)

I have all five anchors fully read. Compiling the structured report.

22. user (2026-06-29T10:51:41.954Z)

Structured output provided successfully